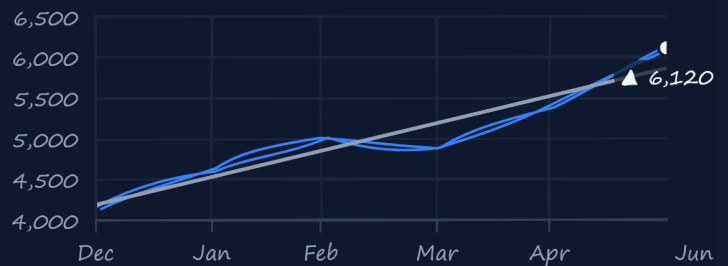


WHY GRAPHEIN

Built for agents, not editors.

Every mainstream chart library — D3, Chart.js, Plotly, even Vega-Lite — assumes a human in an editor: write code, look at the screen, tweak, repeat. A coding agent has **no screen and no edit loop**. Graphein flips it: the agent emits one JSON object and gets machine-readable feedback at every step.

Monthly active users



summarize(spec) → "Users rose 46% from 4,200 to 6,120 between 2024-01 and 2024-06." — deterministic alt-text, no LLM. The peak ▲ is flagged by **insights**; the fit line is a derived **trendline**.

A chart from a library like D3 or Plotly is something a human **looks at**. A Graphein chart is something an agent can **reason about** — generate it as data, prove it's correct, and have it explain itself, with **no human and no vision model in the loop**.

THE AGENT FEEDBACK LOOP

EVERY STEP RETURNS DATA, NOT PIXELS

1

Generate

One object: **type** + a flat **data** array + **encoding**. No imperative API to learn.

2

Validate

validateSpec() — pure, zero-dependency; every error carries a JSON-Patch **fix**.

3

Repair

repairSpec() applies the safe fixes and re-validates in a single call.

4

Render

In a browser, or a headless PNG in CI via **@graphein/node** — no browser needed.

5

Verify

report() flags clipping, overflow & low contrast — the agent checks its work eyes-free.

IT CORRECTS & EXPLAINS ITSELF

LIVE OUTPUT, CAPTURED HEADLESSLY

Self-correcting

VALIDATE → REPAIR

1

The agent emits a typo: `"type": "lyne"`

2

validateSpec() rejects it and returns a machine-applicable fix:

```
[{"op": "replace", "path": "/type", "value": "line"}]
```

3

repairSpec() applies it → `type: "line"`, and re-validation now reports **valid: true**.

Explains itself

NO VISION MODEL

"Users rose 46% from 4,200 to 6,120 between 2024-01 and 2024-06."

summarize(spec) turns any chart into a deterministic one-line description — the agent gets **alt-text without calling an LLM**. The same string powers the chart's **aria-description** for screen-reader accessibility.

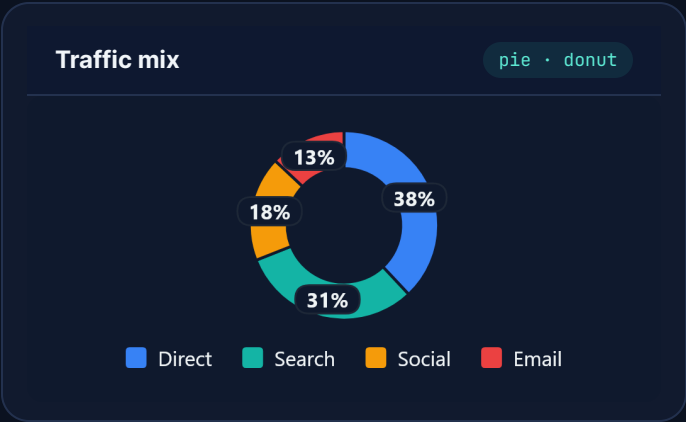
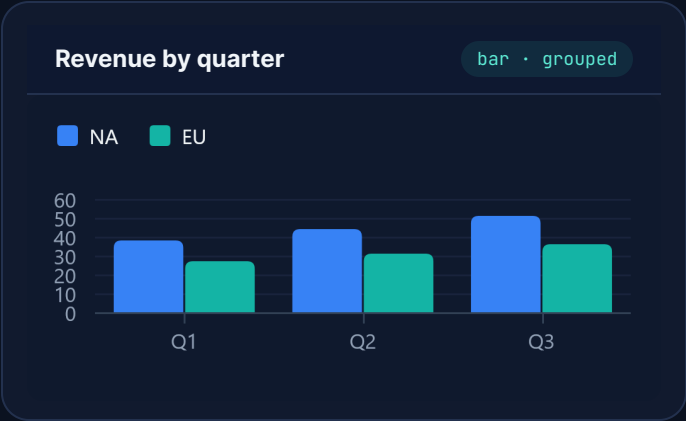
After rendering, **report()** returns a vision-free critique — mark counts, clipped labels, legend overflow, low-contrast colors. If **report.ok** is false, the agent adjusts the spec and re-renders. The critique loop closes without a screenshot.

WHAT AN AGENT NEEDS VS. THE FIELD

	● Graphein	D3	Chart.js	Plotly	Vega-Lite
Spec is plain, serializable JSON	✓ one object	✗ imperative	~ + callbacks	~ config	✓
Pre-render validation with fixable errors	✓ JSON-Patch fix	✗	✗	✗	~ throws
One-call automatic repair	✓ repairSpec()	✗	✗	✗	✗
Post-render critique without a vision model	✓ report()	✗	✗	✗	✗
Deterministic natural-language summary	✓ summarize()	✗	✗	✗	✗
First-class MCP server for tool-use	✓ graphein-mcp	✗	✗	✗	✗

27 CHART TYPES, ONE SHAPE

EVERY TILE BELOW IS REAL GRAPHEIN OUTPUT



Other libraries render a picture. Graphein renders a chart an agent can generate, prove, and explain — start to finish, in JSON.

Provenance: every chart in this document was generated as JSON, validated, and rendered headlessly by Graphein (@graphein/node) — no browser, no vision model. All reported report.ok = true.